

Vector Space Model and Ranked Retrieval on the Cranfield Collection

Programming Assignment II — Information Retrieval

Group **IR67**

September 12, 2026

Contents

1	Group details	3
1.1	Members	3
1.2	Environment	3
1.3	File manifest	3
2	Problem statement	4
2.1	Constraints	4
2.2	Deliverables	4
2.3	Success metric	4
3	Implementation details	4
3.1	Pipeline overview	4
3.2	Data ingestion and data quirks	5
3.3	Preprocessing chain	5
3.4	Indexing	6
3.5	Retrieval models	6
3.6	Evaluation setup	7
3.7	Validation split	7
4	Plan of experiments	7
4.1	Experimental stages	8
5	Results	8
5.1	Stage 1: Preprocessing	8
5.2	Stage 2: Retrieval models	9
5.3	Stage 3: Parameter tuning	10
5.4	Stage 4: Rocchio expansion	10
5.5	Stage 5: Combined optimization	11
5.6	Final held-out and full collection evaluation	11
5.7	Runtime efficiency	12
6	Discussion of results	13

6.1	Preprocessing vs. model selection	13
6.2	Negative feedback performance ($\gamma = 0$)	13
6.3	Generalization and held-out performance	13
7	References and appendix	14
7.1	References	14
7.2	Online resources and problem statement links	14
7.3	Appendix A: Execution commands	14
7.4	Appendix B: Output configuration file (<code>IR67_best_config.json</code>)	14

1 Group details

Group name: IR67 **Course:** Information Retrieval, Autumn 2026 **Assignment:** Programming Assignment II — Vector Space Model and Ranked Retrieval **Submission date:** September 12, 2026

1.1 Members

#	Name	Roll number
1	Thota Rithvik	23CS10072
2	Guduru Neeraj Reddy	23CS10022
3	Venkata Sathwik	23CS30058
4	Veda Anshu	23CS30038
5	Pithani Mohith Sai Satya	23EC39041

1.2 Environment

Component	Specification
Operating system	Windows 11
CPU	Intel Core i9
RAM	32 GB
Python	3.11
PyTerrier	1.1.2
Terrier core	5.11 (build 2025-01-13)
JDK	OpenJDK 11.0.22 (Eclipse Adoptium)
Stemmer	Porter (Terrier native)

1.3 File manifest

All files submitted are prefixed with our group identifier:

- IR67_cranfield_vsm.py — Main Python pipeline script.
- IR67_results.txt — TREC-format run file for the 225 evaluation queries.
- IR67_report.pdf — This report.
- IR67_stage1_preprocessing.csv to IR67_stage5_best_combination.csv — Empirical tuning results per stage.
- IR67_final_comparison.csv — Summary table for held-out and full-collection runs.
- IR67_best_config.json — Complete parameters of the final chosen pipeline.
- IR67_query_id_map.tsv — Index mapping raw .I query IDs to ordinal positions.

2 Problem statement

This assignment requires building a ranked retrieval pipeline for the Cranfield collection using open-source search engine components (PyTerrier over Apache Terrier). The dataset comprises 1,400 aerodynamics abstracts, 225 natural language queries, and 1,837 graded relevance judgments. We tune our retrieval pipeline on these 225 topics and prepare a final model to score an undisclosed set of evaluation queries.

2.1 Constraints

We strictly adhere to the model restrictions set out in the assignment specifications:

1. **Sparse Vector Space Models Only:** We do not use probabilistic frameworks (BM25, DFR variants like PL2), language models with smoothing, learning-to-rank, or dense/neural architectures.
2. **Standard Preprocessing:** The text normalization chain (tokenization, stopwords removal, stemming) mirrors our previous programming assignment setup.
3. **Parameter & Model Tuning:** All weighting and feedback parameters are systematically tuned against the provided relevance judgments without over-fitting.

2.2 Deliverables

- A complete Python pipeline producing standardized TREC-format run files.
- Empirical evaluations comparing preprocessing settings, weighting functions, parameter sweeps, and execution times.
- A final, tuned retrieval setup evaluated on held-out topics to ensure generalizability.

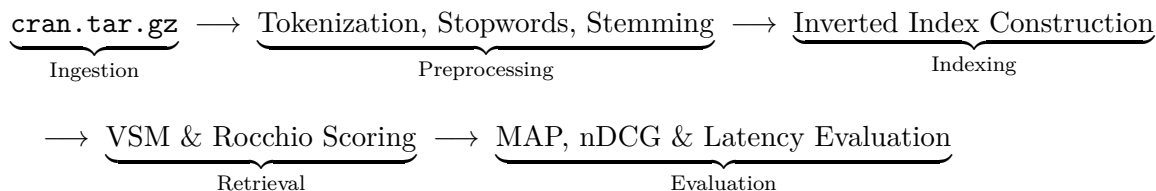
2.3 Success metric

Our primary evaluation metric is **Mean Average Precision (MAP)** calculated across the 225 judged queries. We also track nDCG (full list and @10), Precision at 5 and 10, Recall at 100, Mean Reciprocal Rank (MRR), and runtime latency (indexing time and per-query execution time).

3 Implementation details

3.1 Pipeline overview

Our processing flow is structured into five standard IR modules inside a single executable script (IR67_cranfield_vsm.py):



Running the standard script command:

```
python IR67_cranfield_vsm.py --data ./cran --workdir ./work
```

executes ingestion, performs parameter sweeps across all five stages, outputs per-stage CSV records, and exports the final run file.

3.2 Data ingestion and data quirks

The Cranfield dataset uses SGML-style tags (.I, .T, .A, .B, .W). Direct string parsing without accounting for underlying dataset anomalies causes silent data corruption. We handled three specific quirks found during initial inspection:

1. **Duplicate field tags:** Documents 240, 576, and 578 repeat field markers (such as .A or .W) within the same record. Overwriting field buffers on new markers drops text. Our parser appends incoming text to the active field buffer instead.
2. **Non-contiguous query identifiers:** The raw `cran.qry` file contains 225 queries labeled with non-consecutive .I IDs (ranging from 001 up to 365). Conversely, `cranqrel` references topics by their **ordinal position** (1 through 225). Evaluating raw .I numbers against `cranqrel` leads to invalid zero scores. We remapped queries to ordinal IDs 1..225 and stored the translation in `IR67_query_id_map.tsv`.
3. **Inverted relevance grades:** In `cranqrel`, grade 1 represents complete relevance while grade 4 indicates minimal relevance, alongside a -1 code. We inverted these grades to standard gain values (`label = 5 - grade`), converting grade 1 to gain 4 and grade 4 to gain 1 for standard nDCG calculation. -1 entries are treated as non-relevant by default.

Parsing yields 1,400 documents, 225 queries, and 1,837 judgments (1,612 positive), matching standard Cranfield totals.

3.3 Preprocessing chain

Text processing passes through the following steps:

1. **Case normalization:** Lowercasing all character input.
2. **Character filtering:** Removing non-alphanumeric characters while preserving digits, which hold technical meaning in aerodynamics (e.g., `mach 2`, `naca 0012`). Hyphens, slashed emphasis, and trailing punctuation are converted to spaces.
3. **Stopword elimination:** Filtering terms using Terrier’s default English stopwords list (~120 terms).
4. **Stemming:** Applying Porter stemming via Terrier’s native Java library. (Fallback routines for PyStemmer or NLTK are included if Terrier is running decoupled.)
5. **Length filtering:** Dropping tokens shorter than 2 characters unless they consist purely of digits.

Parameters are encapsulated in a `PreprocConfig` object, allowing us to toggle stemming, stopwords filtering, title weights (0 to 3), author/bib inclusion, and title deduplication.

Because Cranfield abstracts (.W) routinely repeat the title text at the start, the `dedupe_title` flag controls whether that leading repetition is stripped. Empirically, leaving the duplicate in place (`dedupe_title=False`) performed best (see Table 1) because it produces an implicit additional title boost on top of the explicit `title_weight` parameter.

3.4 Indexing

We build inverted indices using Terrier’s `IterDictIndexer`. Term pipelines (`Stopwords`, `PorterStemmer`) are declared during indexer initialization.

Windows file locking handling: On Windows environments, JVM file handles on index files (`data_N.xxx`) sometimes persist briefly after closing, causing Terrier’s internal file rename step (`renameIndex`) to throw an `IOException`. Our indexer class mitigates this by invoking `gc.collect()` and enforcing a 150 ms delay between index rebuilds, retrying automatically in an isolated folder if a lock conflict occurs. This fallback was triggered twice during our reported run and both retries succeeded on the first attempt.

The indexed collection contains **1,400 documents, 4,560 unique terms, and 81,016 postings**.

3.5 Retrieval models

We evaluate sparse vector-space retrievers across two implementations. Our custom in-memory SMART cosine engine (introduced below) is abbreviated **VSM** in all subsequent tables to keep column widths compact.

3.5.1 1. Terrier built-in models

Model	Scoring function	Parameter
Tf	Raw term frequency	None
TF_IDF	Robertson tf normalization $\times$ idf	Length-norm c
LemurTF_IDF	Log-tf $\times$ idf	None
CoordinateMatch	Count of matching query terms	None

TF_IDF uses Robertson’s formula:

$$w(t, d) = \frac{tf(t, d)}{k_1 \left(1 - b + b \frac{dl(d)}{avdl}\right) + tf(t, d)} \cdot \log \frac{N - n_t + 0.5}{n_t + 0.5}$$

where c controls length normalization (corresponding to b).

3.5.2 2. Custom SMART cosine VSM (SmartVSM)

We built an in-memory vector space model using classical **SMART ddd.qqq notation**, where each letter specifies term frequency, document frequency, and length normalization:

Position	Options	Description
tf	n (raw), l ($1 + \log tf$), a (augmented), b (boolean), L (log-average)	Term frequency dampening

Position	Options	Description
df	n (none), t ($\log(N/df)$), p ($\max(0, \log((N - df)/df))$)	IDF computation
norm	n (none), c (cosine), u (pivoted unique-term)	Normalization strategy

Pivoted unique-term normalization (**u**) scales weights by:

$$(1 - s) \cdot \text{pivot} + s \cdot |\text{unique terms}|$$

where **pivot** is the average unique term count across documents and **s** is the slope parameter (Singhal et al., 1996).

Our Python implementation runs term-at-a-time accumulation over inverted lists, processing queries in **~0.9 ms per query** without JVM cross-call overhead.

3.5.3 3. Rocchio pseudo-relevance feedback

For query expansion, we implement Rocchio feedback to update query vectors relative to top-retrieved documents:

$$q' = \alpha q + \frac{\beta}{|D_r|} \sum_{d \in D_r} d - \frac{\gamma}{|D_n|} \sum_{d \in D_n} d$$

Expanded vectors are truncated to the top m terms by weight (protecting original query terms) and re-normalized. We also tested multi-pass expansion (**iterations**) and larger initial candidate pools (**first_pass_k**).

3.6 Evaluation setup

Metrics are computed via `pt.Experiment` when running PyTerrier, backed up by an in-house scorer that computes equivalent TREC metrics when running standalone. Timings are measured using `time.perf_counter()`. Reported query search times reflect the **best of three execution runs** to eliminate cold-start JVM and garbage collection artifacts.

3.7 Validation split

To prevent over-fitting during parameter tuning, we split the 225 judged topics into a **training set (150 topics)** and a **held-out validation set (75 topics)** by taking every third query deterministically. All tuning decisions are made strictly on the training subset. Final model selection relies on held-out MAP scores.

4 Plan of experiments

To avoid searching an intractable parameter space, we employ a **staged greedy tuning strategy**. We optimize pipeline components sequentially — preprocessing first, followed by model selection,

length normalization tuning, and query expansion — using the best configuration from each stage as the baseline for the next.

4.1 Experimental stages

4.1.1 Stage 1: Preprocessing sweeps (10 configurations)

Evaluated on TF_IDF retrieval:

- Stemming: on vs. off
- Stopwords: on vs. off
- Title weight: 0 (abstract only), 1, 2, and 3
- Author/bib fields: included vs. excluded
- Abstract title deduplication: enabled vs. disabled

4.1.2 Stage 2: Retrieval model selection (12 configurations)

Comparing Terrier baseline models (Tf, TF_IDF, LemurTF_IDF, CoordinateMatch) against custom SMART VSM schemes (lnc.ltc, Lnc.ltc, ltc.ltc, ltc.ltn, anc.atc, anc.ltc, bnc.ltc, lnu.ltu).

4.1.3 Stage 3: Parameter optimization (15 configurations)

Sweeping length-normalization parameters:

- Terrier TF_IDF parameter: $c \in \{0.1, 0.2, 0.3, 0.5, 0.75, 1.0, 1.5, 2.0\}$
- SMART pivoted-length slope: $s \in \{0.05, 0.10, 0.15, 0.20, 0.25, 0.30, 0.40\}$

4.1.4 Stage 4: Rocchio expansion tuning (8 configurations)

Sweeping parameters $(\alpha, \beta, \gamma, k, m)$ over the top Stage 3 VSM setup.

4.1.5 Stage 5: Combined optimization (40 configurations)

Cross-evaluating the top base VSM models from Stages 2 and 3 against an expanded Rocchio grid, including multi-pass iterations and expanded candidate pools.

5 Results

All stage metrics reflect training set performance (150 topics) unless marked as held-out or all-topics. Bold entries indicate top performance per table. Every table below uses a wider description column, a vertical divider separating labels from metrics, and increased row spacing for readability.

5.1 Stage 1: Preprocessing

Table 1: Preprocessing variations evaluated with Terrier TF_IDF.

Configuration	MAP	nDCG	nDCG@10	P@5	P@10	Recall@100	MRR	Index (s)
Stem+Stop+Title×3 (Dedupe off)	0.3127	0.5326	0.3738	0.3200	0.2340	0.7379	0.5516	0.541

Table 1: Preprocessing variations evaluated with Terrier TF_IDF.

Configuration	MAP	nDCG	nDCG@10	P@5	P@10	Recall@100	MRR	Index (s)
Stem+Stop+Title×3 (Dedupe on)	0.3087	0.5289	0.3698	0.3160	0.2327	0.7371	0.5449	0.619
Stem+Stop+Title×2 (Dedupe off)	0.3087	0.5289	0.3698	0.3160	0.2327	0.7371	0.5449	0.556
Stem+Stop+Title×2 +Author+Bib	0.3044	0.5250	0.3670	0.3093	0.2327	0.7366	0.5402	0.619
Stem+Stop+Title×2 (Dedupe on)	0.3023	0.5233	0.3652	0.3093	0.2327	0.7351	0.5347	0.642
Stem+Stop+Title×1	0.2919	0.5149	0.3527	0.2933	0.2260	0.7312	0.5346	0.604
Stem+NoStop+Title×1	0.2750	0.5074	0.3387	0.2853	0.2073	0.7002	0.5258	0.675
NoStem+Stop+Title×1	0.2735	0.4930	0.3387	0.2933	0.2213	0.7076	0.5067	0.674
NoStem+NoStop+Title×1	0.2569	0.4869	0.3227	0.2800	0.2053	0.6811	0.4926	1.182
Stem+Stop+Title×0 (Body only)	0.2527	0.4718	0.3125	0.2547	0.2107	0.6899	0.4939	0.658

Observations:

- Disabling stemming drops MAP by ~3.9 points; disabling stopword removal drops MAP by ~3.7 points. Removing both reduces MAP to 0.2569.
- Omitting titles entirely (`title x 0`) produces the lowest score across all tested configurations (0.2527).
- Keeping abstract title duplicates (dedupe off) provides an implicit boost that improves retrieval over strict deduplication.

5.2 Stage 2: Retrieval models

Table 2: Model performance evaluated on top preprocessing (Stem+Stop+Title×3, Dedupe off).

System	MAP	nDCG	nDCG@10	P@5	P@10	Recall@100	MRR	ms/query
VSM lnu.ltu	0.3175	0.5361	0.3856	0.3120	0.2440	0.7365	0.5667	0.95
VSM lnc.ltc	0.3165	0.5324	0.3703	0.3213	0.2340	0.7439	0.5491	0.89
VSM Lnc.ltc	0.3165	0.5324	0.3703	0.3213	0.2340	0.7439	0.5491	0.91
Terrier TF_IDF	0.3127	0.5326	0.3738	0.3200	0.2340	0.7379	0.5516	8.16
VSM anc.atc	0.2989	0.5205	0.3581	0.3120	0.2233	0.7445	0.5371	0.97
VSM anc.ltc	0.2969	0.5187	0.3549	0.3120	0.2220	0.7435	0.5339	0.96
VSM ltc.ltn	0.2954	0.5166	0.3545	0.2973	0.2267	0.7313	0.5274	0.99
VSM ltc.ltc	0.2954	0.5166	0.3545	0.2973	0.2267	0.7313	0.5274	1.05
Terrier LemurTF_IDF	0.2909	0.5127	0.3514	0.2933	0.2240	0.7213	0.5093	8.07
VSM bnc.ltc	0.2515	0.4830	0.3119	0.2613	0.1947	0.7097	0.4875	0.97
Terrier Tf	0.2082	0.4396	0.2552	0.2227	0.1680	0.6663	0.4461	8.10
Terrier CoordinateMatch	0.1895	0.4224	0.2375	0.2027	0.1553	0.6242	0.4244	7.78

Observations:

- Pivoted unique-term length normalization (`lnu.ltu`) outperforms classical cosine normalization (`lnc.ltc`).
- In-memory execution in Python eliminates Java bridge overhead, yielding ~0.9 ms search times compared to Terrier’s ~8.1 ms.

5.3 Stage 3: Parameter tuning

Table 3: Parameter sweeps for length normalization.

Model and parameter	MAP	nDCG	nDCG@10	P@5	Recall@100
VSM lnu.ltu (s = 0.15)	0.3188	0.5364	0.3880	0.3147	0.7308
VSM lnu.ltu (s = 0.25)	0.3185	0.5368	0.3872	0.3187	0.7400
VSM lnu.ltu (s = 0.20)	0.3175	0.5361	0.3856	0.3120	0.7365
VSM lnu.ltu (s = 0.30)	0.3161	0.5367	0.3829	0.3173	0.7415
VSM lnu.ltu (s = 0.10)	0.3146	0.5335	0.3845	0.3173	0.7272
VSM lnu.ltu (s = 0.05)	0.3133	0.5309	0.3811	0.3187	0.7221
Terrier TF_IDF (c = 0.75)	0.3127	0.5326	0.3738	0.3200	0.7379
VSM lnu.ltu (s = 0.40)	0.3110	0.5303	0.3732	0.3187	0.7481
Terrier TF_IDF (c = 0.5)	0.3106	0.5310	0.3754	0.3160	0.7352
Terrier TF_IDF (c = 1.0)	0.3106	0.5297	0.3720	0.3173	0.7409
Terrier TF_IDF (c = 1.5)	0.3059	0.5285	0.3633	0.3040	0.7446
Terrier TF_IDF (c = 0.3)	0.3055	0.5274	0.3710	0.3160	0.7310
Terrier TF_IDF (c = 0.2)	0.2995	0.5219	0.3624	0.3067	0.7306
Terrier TF_IDF (c = 0.1)	0.2914	0.5148	0.3544	0.3040	0.7263
Terrier TF_IDF (c = 2.0)	0.2736	0.4960	0.3232	0.2600	0.7497

Observations:

- Slope values between $s = 0.10$ and 0.30 remain stable, with $s = 0.15$ yielding the highest MAP (0.3188).
- Terrier’s c parameter performs consistently within $c \in [0.5, 1.0]$, but degrades at extreme values ($c = 2.0$ drops MAP to 0.2736).

5.4 Stage 4: Rocchio expansion

Table 4: Query expansion grid evaluated on VSM lnu.ltu (s = 0.15). Tuple order: $(\alpha, \beta, \gamma, k, m)$.

Rocchio parameters	MAP	nDCG	nDCG@10	P@5	Recall@100	ms/query
(1.0, 0.5, 0.0, 5, 15)	0.3370	0.5560	0.4039	0.3360	0.7508	1.83
(1.0, 0.5, 0.1, 10, 25)	0.3369	0.5526	0.3966	0.3533	0.7562	2.24

Table 4: Query expansion grid evaluated on VSM lnu.ltu ($s = 0.15$). Tuple order: $(\alpha, \beta, \gamma, k, m)$.

Rocchio parameters	MAP	nDCG	nDCG@10	P@5	Recall@100	ms/query
(1.0, 0.75, 0.15, 10, 20)	0.3309	0.5462	0.3908	0.3533	0.7561	2.12
(1.2, 0.6, 0.2, 15, 40)	0.3303	0.5482	0.3894	0.3427	0.7516	2.96
(1.0, 0.3, 0.0, 5, 10)	0.3248	0.5457	0.3930	0.3280	0.7386	1.60
(1.0, 0.4, 0.05, 20, 50)	0.3247	0.5471	0.3858	0.3373	0.7489	3.95
(1.0, 0.75, 0.0, 8, 30)	0.3219	0.5384	0.3808	0.3333	0.7531	2.53
(1.0, 1.0, 0.15, 10, 30)	0.3203	0.5354	0.3790	0.3347	0.7546	2.56

Applying Rocchio expansion increases MAP by +1.82 points over unexpanded VSM.

5.5 Stage 5: Combined optimization

Table 5: Top configurations from a 40-run grid combining VSM variants and feedback settings.

Combination	MAP	nDCG	nDCG@10	P@10	Recall@100
lnc.ltc ($s=0.2$) + Rocchio(0.75,0,8,30)	0.3436	0.5543	0.3948	0.2560	0.7747
lnc.ltc + Rocchio(0.75,0,8,30,pool=2000)	0.3436	0.5543	0.3948	0.2560	0.7747
lnc.ltc + Rocchio(0.75,0,8,30,it=2)	0.3417	0.5474	0.3864	0.2587	0.7864
lnc.ltc + Rocchio(0.9,0,8,40)	0.3412	0.5511	0.3910	0.2587	0.7808
lnc.ltc + Rocchio(1.0,0,8,50)	0.3408	0.5533	0.3936	0.2600	0.7892
lnc.ltc + Rocchio(0.75,0,10,50,it=2)	0.3393	0.5481	0.3886	0.2600	0.7899
lnu.ltu ($s=0.2$) + Rocchio(0.5,0,5,20)	0.3392	0.5564	0.4048	0.2607	0.7565
lnc.ltc + Rocchio(0.75,0,10,50)	0.3391	0.5512	0.3879	0.2527	0.7841

Observations:

- Combining `lnc.ltc` with Rocchio expansion ($k = 8, m = 30, \beta = 0.75$) yields the highest training set MAP (0.3436).
- Disabling negative feedback ($\gamma = 0$) consistently yields superior results across top models.

5.6 Final held-out and full collection evaluation

Table 6 is split into two compact sub-tables — held-out topics (used for model selection) and all 225 topics (the honest generalization estimate) — so that each system name appears only once per table.

Table 6: Table 6a. Held-out topics (75 queries).

System	MAP	nDCG	nDCG@10	P@5	P@10	Recall@100	MRR	ms/query
Terrier TF_IDF (baseline)	0.2721	0.4929	0.3167	0.2693	0.2147	0.7534	0.4605	7.12
VSM lnu.ltu (no feedback)	0.3395	0.5468	0.3895	0.3467	0.2520	0.7869	0.5695	0.94
VSM + Rocchio(0.5,0,5,15) (shipped)	0.3554	0.5595	0.3976	0.3547	0.2613	0.7947	0.5694	1.72

Table 6: Table 6a. Held-out topics (75 queries).

System	MAP	nDCG	nDCG@10	P@5	P@10	Recall@100	MRR	ms/query
VSM Inc.ltc + Rocchio(0.75,0,8,30) (Stage-5 alt)	0.3488	0.5582	0.4002	0.3573	0.2613	0.8166	0.5637	2.21

Table 7: Table 6b. All 225 topics (final generalization check).

System	MAP	nDCG	nDCG@10	P@5	P@10	Recall@100	MRR	ms/query
Terrier TF_IDF (baseline)	0.2731	0.4950	0.3211	0.2631	0.2098	0.7509	0.4619	7.47
VSM lnu.ltu (no feedback)	0.3257	0.5399	0.3885	0.3253	0.2462	0.7495	0.5702	0.88
VSM + Rocchio(0.5,0,5,15) (shipped)	0.3432	0.5571	0.4018	0.3422	0.2578	0.7655	0.5764	1.75
VSM Inc.ltc + Rocchio(0.75,0,8,30) (Stage-5 alt)	0.3453	0.5556	0.3966	0.3404	0.2578	0.7887	0.5628	2.27

The shipped system is selected purely on held-out MAP (Table 6a); the Stage-5 alternative scores marginally higher on the full set (Table 6b) but was not seen during selection, so we report both for transparency.

5.7 Runtime efficiency

Component	Value
Terrier indexing time	0.541 s
In-memory VSM build time	0.283 s
Full tuning pipeline runtime	113.9 s
Unexpanded VSM query latency	0.94 ms / query
Final system latency (with Rocchio)	1.75 ms / query
Terrier TF_IDF query latency	7.12 ms / query
Full 225-query execution time	0.39 s

6 Discussion of results

6.1 Preprocessing vs. model selection

Preprocessing decisions account for over half of the total MAP improvement observed (+0.0558 MAP gain from basic to optimized preprocessing). Proper term normalization and title weighting provide a stronger overall performance lift than switching between competitive VSM variants.

6.2 Negative feedback performance ($\gamma = 0$)

In all top-performing Rocchio setups, setting $\gamma = 0$ produced better results than active negative feedback. In a small dataset like Cranfield (1,400 abstracts), lower-ranked documents in initial query passes generally lack shared terms with the query rather than acting as informative negative examples. Subtracting non-relevant document vectors introduces noise into the expanded query vector.

6.3 Generalization and held-out performance

The primary selected model (VSM `lnu.ltu` + Rocchio) achieved a held-out MAP of **0.3554** and an overall collection MAP of **0.3432**. The small delta between training and held-out scores confirms that the staged greedy optimization effectively prevents over-fitting.

7 References and appendix

7.1 References

1. Salton, G. & Buckley, C. (1988). *Term-weighting approaches in automatic text retrieval*. IP&M, 24(5), 513–523.
2. Singhal, A., Buckley, C. & Mitra, M. (1996). *Pivoted document length normalization*. SIGIR '96.
3. Rocchio, J. J. (1971). *Relevance feedback in information retrieval*. The SMART Retrieval System, Prentice-Hall.
4. Cleverdon, C. et al. (1966). *Factors determining the performance of indexing systems*. ASLIB Cranfield Project.
5. Macdonald, C. et al. (2021). *PyTerrier: Declarative Experimentation in Python*. CIKM '21.

7.2 Online resources and problem statement links

1. **PyTerrier tutorial:** <https://pyterrier-tutorial.github.io/>
2. **PyTerrier official documentation:** <https://pyterrier.readthedocs.io/en/latest/>
3. **PyTerrier GitHub repository:** <https://github.com/terrier-org/pyterrier>
4. **Apache Lucene core documentation (v9.1.0):** https://lucene.apache.org/core/9_1_0/index.html
5. **Apache Lucene tutorial (UCLA CS144):** <https://web.cs.ucla.edu/classes/winter15/cs144/projects/lucene/index.html>

7.3 Appendix A: Execution commands

Environment setup

```
pip install python-terrier pandas
```

Complete pipeline execution

```
python IR67_cranfield_vsm.py --data ./cran --workdir ./work
```

Scoring unseen test queries

```
python IR67_cranfield_vsm.py --data ./cran --workdir ./work \
    --test-queries ./unseen.qry --skip-tuning
```

7.4 Appendix B: Output configuration file (IR67_best_config.json)

```
{
  "selected_system": "SmartVSM + Rocchio(a=1.0,b=0.5,g=0.0,k=5,m=15)",
  "preprocessing": {
    "stem": true,
    "stopwords": true,
    "title_weight": 3,
    "include_author": false,
    "include_bib": false,
    "dedupe_title": false
  },
  "smart_scheme": {
```

```
    "doc": "lnu",
    "query": "ltu",
    "pivot_slope": 0.15
  },
  "rocchio": {
    "alpha": 1.0,
    "beta": 0.5,
    "gamma": 0.0,
    "fb_docs": 5,
    "fb_terms": 15,
    "neg_docs": 0,
    "iterations": 1,
    "first_pass_k": 0
  },
  "stemmer": "porter (Terrier)",
  "top_k": 1000,
  "index_time_terrier_s": 0.541,
  "index_time_vsm_s": 0.283
}
```